Studio Nx

JS 117 — divisionTicks()

2026-10-14

JS 117 — `divisionTicks()` (스냅 칸 이름을 tick 길이로)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 세 번째 함수 (FR-206 그리드 스냅)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	셋잇단(1/n 의 3등분) 칸 길이 함수, 그리고 Snap 메뉴 연결을 이어서 진행합니다

1. 하는 일

상단 Snap 메뉴에서 고르는 **칸 이름**(`1/1` ~ `1/32`)을 그 칸 하나의 **tick 길이**로 바꿉니다. FR-206의 그리드 스냅에서 "몇 tick마다 붙을지"를 정하는 값입니다.

> 1/n 음표의 tick = PPQ(960) × 4 ÷ n (음표 길이라 박자표와 무관합니다)

칸	한 칸 tick	마디로(4/4)
`1/1` (온음표)	3840	1마디
`1/2` (2분음표)	1920	0.5마디
`1/4` (4분음표)	960	0.25마디
`1/8`	480	0.125마디
`1/16`	240	0.0625마디
`1/32`	120	0.03125마디
목록에 없는 이름(`1/3`, `1/64`, 숫자 등)	오류(`RangeError`)	—

2. 코드

```
const DIVISION_DEN = { '1/1': 1, '1/2': 2, '1/4': 4, '1/8': 8, '1/16': 16, '1/32': 32 };

export function divisionTicks(division) {
  const den = DIVISION_DEN[division];
  if (typeof den !== 'number') throw new RangeError('알 수 없는 스냅 칸입니다: ' + division);
  return (PPQ * 4) / den;
}
```

최신 문법으로 쓰면 (공부용)

```
const DIVISION_DEN = { '1/1': 1, '1/2': 2, '1/4': 4, '1/8': 8, '1/16': 16, '1/32': 32 };
export function divisionTicks(division) {
  const den = DIVISION_DEN[division];
  if (!Number.isInteger(den)) throw new RangeError(`알 수 없는 스냅 칸입니다: ${division}`);
  return (PPQ * 4) / den;
}
```

최신 문법	실제 코드	뜻
`Number.isInteger(x)`	`typeof x !== 'number'`	값이 정수인지 확인합니다. 아래 3번처럼 상속된 키를 걸러 내는 데도 씁니다
템플릿 문자열	`'…' + x`	글자와 값을 잇습니다

3. 왜 이렇게

선택	이유
음표 길이를 tick으로 돌려줌	음표 길이는 박자표와 무관합니다(`1/4`는 4/4·3/4 모두 1박 = 960). 마디로 바꾸는 일은 `ticksToBars()`가 맡아 역할을 나눕니다
표(객체)로 분모를 둠	960÷n을 매번 계산하지 않고, 허용된 칸만 받도록 합니다. 화면 Snap 목록과 같은 값을 씁니다
`typeof den !== 'number'`로 검사	`division in 표`나 `표[division]`만 보면 `toString`·`constructor` 같은 **상속된 키**가 통과합니다. 숫자일 때만 돌려주어 막습니다
잘못된 값은 오류를 던짐	없는 칸으로 스냅하면 위치가 조용히 틀어집니다. 바로 알리는 쪽이 낫습니다
셋잇단은 다음 함수	셋잇단 칸은 "1/n을 3등분"이라 규칙이 따로입니다. 한 함수씩 나눠 검토합니다

4. 시험 (Node에서 직접 실행, 15개 통과)

확인	결과
`1/1`~`1/32`가 3840·1920·960·480·240·120 tick	✓
`1/4`·`1/16`을 마디로 바꾸면 0.25·0.0625	✓
없는 칸(`1/3`·`1/64`·빈 문자열), 숫자·없음 → 오류	✓
상속된 키(`toString`·`constructor`) → 오류	✓